Guide de soutenance — Propagation d'impact et notification optimisée

Ce que tu dois savoir expliquer : comment tout tester, la structure, chaque choix, le déroulement complet et le détail de chaque test.

1. Le projet en 30 secondes

Le problème. Dans un événement avec plusieurs ateliers, quand l'organisateur change quelque chose (horaire, salle, annulation), il faut prévenir uniquement les personnes réellement concernées, dans le bon ordre, sans dépasser le débit autorisé par l'API d'envoi, et sans envoyer trois messages contradictoires si l'organisateur change trois fois d'avis en deux minutes.

La solution, en quatre briques :

1. **Qui est impacté ?** Un parcours de graphe (BFS) suit les dépendances entre ateliers (Tâche 3).
2. **Quoi leur dire ?** Les changements rapprochés sont fusionnés : une seule notification cohérente par personne (Tâche 7).
3. **Dans quel ordre ?** Une file de priorité envoie les annulations avant les changements d'horaire, avant les infos mineures (Tâche 5).
4. **À quel rythme ?** Un limiteur de débit à fenêtre glissante empêche de dépasser N envois par période (Tâche 6).

La phrase à retenir : « Mon programme décide qui prévenir (graphe), quoi leur dire (fusion), dans quel ordre (priorité) et à quel rythme (débit). »

Ce qui est simulé (à dire avant qu'on te le demande) : l'envoi réel d'un SMS ou d'un email n'existe pas, il est représenté par une fonction `send_fn` remplaçable. Le temps est simulé dans les tests par une horloge artificielle. Et rien ne déclenche tout seul l'émission des groupes : c'est l'appelant qui doit appeler `flush_ready()` (voir la section 9).

2. Comment tout tester

2.1 Installation depuis zéro

```
git clone https://github.com/TimotheeO/NOM_DU_REPO.git
cd NOM_DU_REPO
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
```

Sous Windows, l'activation se fait avec le script `activate` du dossier `venv/Scripts`. Une fois l'environnement actif, `(venv)` apparaît au début de la ligne de commande.

2.2 Les commandes de test

Commande	Ce que ça fait
`python3 -m pytest`	Lance les 92 tests, affiche chaque nom et les affichages visuels
`python3 -m pytest tests/test_batching.py`	Un seul fichier de tests
`python3 -m pytest -k cancellation`	Seulement les tests dont le nom contient « cancellation »
`python3 -m pytest -q -o addopts=`	Version silencieuse : juste les points et le total
`ruff check .`	Vérifie le style du code (le même contrôle que la CI)

Résultat attendu : `92 passed` pour pytest et `All checks passed!` pour ruff.

2.3 Lire la sortie de pytest

- `PASSED` (vert) : le test est réussi. `FAILED` (rouge) : pytest affiche la ligne du `assert` qui a échoué et les deux valeurs comparées.
- Le fichier `pytest.ini` contient `-v -rP`. `-v` affiche un nom par test. `-rP` affiche aussi les `print` des tests réussis (par défaut pytest les cache). C'est pour ça que tu vois l'histogramme du débit et le rapport de fusion sans aucune option.
- Un nom comme `test_limit_respected_for_many_configurations[5-1.0-120]` est un même test lancé avec plusieurs jeux de paramètres (`parametrize`) : ici limite 5, période 1,0 s, 120 notifications.

2.4 Les démos : voir le résultat concret

Commande	Ce que tu vois
`python3 demo.py w1`	Qui est impacté si Cuisine change : 3 personnes sur 5
`python3 demo.py w3`	Si Poterie change : David et Emma
`python3 demo_pipeline.py`	Partie 1 : 4 changements fusionnés en 5 notifications. Partie 2 : 120 notifications à 5 par seconde

2.5 Prouver que les tests ne sont pas « vides »

Un test qui ne peut jamais échouer ne prouve rien. Devant le jury, tu peux casser le code exprès :

1. Dans `src/rate_limiter.py`, méthode `try_acquire`, remplace `if len(self._sent_at) < self.max_per_period:` par `if True:` (le limiteur laisse alors tout passer).
2. Lance `python3 -m pytest` : plusieurs tests passent au rouge, et le test principal du débit affiche « LIMITE DÉPASSÉE ».
3. Restaure le fichier avec `git checkout src/rate_limiter.py`.

C'est exactement ce que j'ai fait pour valider le code : j'ai cassé le limiteur, la fusion (dernière valeur gagnante) et la règle d'annulation, et à chaque fois des tests ont échoué.

2.6 Ce que fait la CI GitHub

À chaque push, `.github/workflows/ci.yml` installe Python 3.12 et les dépendances, lance `ruff check .` puis `pytest tests/ -v`. Un push n'est vert que si les deux passent. Le lint a déjà échoué une fois (imports non triés) : preuve que la CI fait son travail.

3. Structure du projet

```
CalculProg-Impact-NotifOpti/
├── .github/workflows/ci.yml   CI : ruff + pytest à chaque push
├── data/sample_event.json     événement de test (5 personnes, 4 ateliers)
├── docs/SCHEMA.md             schéma du graphe
├── docs/PIPELINE.md           schéma de la chaîne de notification
├── src/
│   ├── graph.py               modèle du graphe
│   ├── data_loader.py         JSON → graphe
│   ├── impact.py              BFS : qui est impacté
│   ├── notifications.py       niveaux d'urgence + file de priorité
│   ├── batching.py            fusion des changements rapprochés
│   ├── rate_limiter.py        limiteur de débit
│   ├── dispatcher.py          file de priorité + limiteur → envoi
│   ├── clock.py               horloge simulée
│   ├── report.py              affichages du débit + vérificateur
│   └── batching_report.py     affichage de la fusion
├── tests/                     conftest.py + 8 fichiers de tests
├── demo.py, demo_pipeline.py  démos en ligne de commande
└── pytest.ini, requirements.txt, .gitignore
```

3.1 Qui dépend de qui

```
graph.py  ◄── data_loader.py
graph.py  ◄── impact.py (utilise les méthodes du graphe)
impact.py ◄── batching.py ──► notifications.py
notifications.py ◄── dispatcher.py ──► rate_limiter.py
dispatcher.py ◄── report.py          batching.py ◄── batching_report.py
clock.py : indépendant, injecté partout où le temps compte
```

3.2 Pourquoi cette structure

- **Un module = une responsabilité.** Si on te demande « où est gérée la priorité ? », la réponse tient en un fichier : `notifications.py`.
- **Les dépendances vont dans un seul sens**, jamais en cercle : le graphe ne connaît ni les notifications ni le temps.
- **Injection de dépendances.** L'horloge (`clock`), la fonction d'attente (`sleep`) et la fonction d'envoi (`send_fn`) sont passées en paramètre. En production on met `time.monotonic`, `time.sleep` et un vrai appel d'API ; dans les tests on met l'horloge simulée.
- **Structure miroir pour les tests** : un fichier `test_xxx.py` par module `xxx.py`, plus `test_pipeline.py` pour la chaîne complète.

4. Le déroulement complet, pas à pas

Scénario de `demo_pipeline.py`, partie 1. Fenêtre de regroupement : 30 s. Limite d'envoi : 2 par seconde. L'événement :

```
w1 Cuisine  (Alice, Bruno)  ──dépend──▶  w2 Dégustation (Bruno, Chloe)
w3 Poterie  (David)         ──dépend──▶  w4 Vernissage   (Emma)
```

« w2 dépend de w1 » signifie : si Cuisine change, Dégustation est impactée en cascade, mais pas l'inverse.

Étape 0 : chargement. `load_event_graph` lit le JSON et remplit un `EventGraph` : deux dictionnaires (participants et ateliers). Chaque atelier porte deux ensembles : ses participants et les ateliers qui dépendent de lui.

Étape 1 : t = 0 s, l'horaire de Cuisine passe à 15h. `batcher.register_change('w1', 'horaire', '15h', CHANGEMENT_HORAIRE)`.

- `compute_impacted_set` fait un BFS : on part de w1, ses dépendants sont {w2}, ceux de w2 sont {} → ateliers impactés {w1, w2}. Participants = {Alice, Bruno} ∪ {Bruno, Chloe} = {Alice, Bruno, Chloe}. Bruno n'est compté qu'une fois (c'est un ensemble).
- Pour chacun, le batcher crée un groupe en attente (`opened_at = 0`) contenant `(w1, horaire) → ('15h', CHANGEMENT_HORAIRE)`.
- Compteur « notifications sans fusion » : +3.

Étape 2 : t = 5 s, l'horaire passe à 16h. Mêmes trois personnes, groupes déjà existants. La clé `(w1, horaire)` existe, sa valeur est écrasée par `16h`. Aucun nouveau groupe, `opened_at` reste 0. Compteur sans fusion : 6.

Étape 3 : t = 12 s, la salle devient B (info mineure). Nouvelle clé `(w1, salle)` ajoutée aux trois groupes. Compteur : 9.

Étape 4 : t = 15 s, Poterie est annulée. `register_change('w3', 'statut', 'annulé', ANNULATION)`. BFS : w3 → w4, donc David et Emma. Deux nouveaux groupes (`opened_at = 15`). Compteur : 11.

Étape 5 : t = 45 s, émission. `flush_ready()` regarde chaque groupe : son âge est-il supérieur ou égal à 30 s ? Alice, Bruno, Chloe : 45 − 0 = 45, oui. David, Emma : 45 − 15 = 30, oui. Pour chaque groupe, `_build_notification` fait trois choses :

1. Il cherche les ateliers annulés (clé `statut` avec la valeur `annulé`) et retire leurs autres changements.
2. L'urgence finale est la plus haute (la valeur numérique la plus basse) parmi les changements restants.
3. Il assemble UN seul message.

Résultat : Alice, Bruno et Chloe reçoivent « Atelier Cuisine : horaire → 16h, salle → B » en CHANGEMENT_HORAIRE (le minimum de 1 et 2). David et Emma reçoivent « Atelier Poterie : annulé » en ANNULATION. **11 notifications sans fusion → 5 émises, 6 évitées.**

Étape 6 : la file de priorité. Les 5 notifications sont poussées dans `NotificationQueue`. En interne, c'est un tas (`heapq`) de tuples `(urgence, compteur, notification)`. Les ANNULATION (valeur 0) sortiront avant les CHANGEMENT_HORAIRE (valeur 1).

Étape 7 : envoi sous contrainte de débit. `dispatcher.drain()` boucle :

- `dispatch_available()` : tant que la file n'est pas vide ET que `limiter.try_acquire()` répond oui, il sort la plus urgente et l'envoie. À t = 45, David et Emma partent (2 envois), puis le troisième `try_acquire` répond non (déjà 2 envois dans la dernière seconde).
- La file n'est pas vide : `seconds_until_available()` calcule 1,0 s (le plus ancien envoi quitte la fenêtre à t = 46). Le dispatcher dort 1 s (dans les tests, l'horloge simulée avance de 1 s).
- À t = 46 : deux envois. À t = 47 : le dernier.
- Envois à 45, 45, 46, 46, 47. Jamais plus de 2 sur une seconde.

Étape 8 : vérification. `max_in_any_window` recompte indépendamment les envois sur toutes les fenêtres d'une seconde : maximum observé 2, limite 2, verdict OK.

5. Les choix techniques et leur justification

Choix	Pourquoi	Alternative écartée
Python	Structures natives adaptées (`set`, `deque`, `heapq`), zéro configuration, lisible à l'oral	Java ou TypeScript : plus de plomberie pour le même résultat
Graphe orienté en listes d'adjacence (ensembles)	Test d'appartenance en O(1), doublons impossibles, mémoire proportionnelle au nombre de relations	Matrice d'adjacence : O(n²) en mémoire, inutile pour un graphe peu dense
Dépendance orientée (base → dépendant)	Si Cuisine change, Dégustation est impactée, pas l'inverse	Graphe non orienté : on notifierait trop de monde
BFS avec `deque` et ensemble `visited`	O(W+E), gère cycles et chemins multiples, `popleft` en O(1)	DFS : même résultat, autre ordre de parcours. `list.pop(0)` : O(n)

Choix	Pourquoi	Alternative écartée
`heapq` pour la file de priorité	Tas binaire de la bibliothèque standard : ajout et retrait en O(log n), rien à maintenir	Trier la liste à chaque ajout, ou coder son propre tas
`IntEnum` avec ANNULATION = 0	`heapq` sort le plus petit : le plus urgent a donc la plus petite valeur	Entiers magiques ou valeurs négatives
Compteur `itertools.count()` dans le tas	Départage les urgences égales (ordre d'arrivée) et évite de comparer deux `Notification`, qui ne sont pas comparables	Rien : plantage dès que deux urgences sont égales
Fenêtre glissante pour le débit	Garantit strictement « jamais plus de N sur n'importe quelle période »	Token bucket : autorise des rafales, donc peut dépasser N sur une seconde glissante
Horloge injectée (`SimulatedClock`)	Tests instantanés et déterministes : 30 s de fenêtre testées en quelques millisecondes	`time.sleep` dans les tests : lents et instables
Groupes par participant	Une notification redondante, c'est une même personne touchée plusieurs fois : un groupe par personne les détecte naturellement	Grouper par atelier : une personne inscrite à deux ateliers recevrait deux messages
Fenêtre fixe depuis le premier changement	Le délai maximal est borné (30 s)	Debounce pur : relancé à chaque changement, il peut retarder indéfiniment
La dernière valeur gagne	15h puis 16h : seule 16h est vraie	Garder les deux : message contradictoire
L'annulation remplace les autres changements	« Horaire décalé » pour un atelier qui n'a plus lieu est absurde	Tout lister
Urgence finale = la plus haute	Une personne ne doit pas rater une annulation parce qu'un autre changement est mineur	Prendre la dernière urgence : une info mineure tardive masquerait l'annulation

Choix	Pourquoi	Alternative écartée
Dispatcher séparé de la file et du limiteur	Chaque classe a une seule responsabilité et se teste seule	Tout mettre dans `NotificationQueue`
Vérificateur de débit indépendant	`max_in_any_window` recompte lui-même les envois : il détecterait un bug du limiteur	Tester le limiteur avec lui-même : test circulaire
Un fichier de test par module, plus `conftest.py`	Structure miroir et fixtures partagées (`clock` , `sample_graph`)	Un seul gros fichier de tests

6. Le code, fichier par fichier

6.1 `src/graph.py` : le modèle du graphe

- **`Participant`** et **`Workshop`** sont des `@dataclass` . Le décorateur génère automatiquement `__init__` , `__repr__` et `__eq__` . On écrit `participant.name` plutôt que `participant['name']` : une faute de frappe est détectée tout de suite et l'éditeur complète.
- **`Workshop`** porte deux ensembles : `participant_ids` (les inscrits) et `dependent_workshop_ids` (les ateliers qui dépendent de lui). On utilise `field(default_factory=set)` et pas `= set()` , car en Python une valeur par défaut mutable serait créée une seule fois et **partagée par tous les ateliers**. `default_factory` appelle `set()` à chaque nouvel objet.
- **`EventGraph`** contient deux dictionnaires `id → objet` : `participants` et `workshops` .
  - `add_participant` et `add_workshop` créent l'objet, le rangent dans le dictionnaire et le retournent.
  - `enroll(participant_id, workshop_id)` vérifie que les deux existent (sinon `ValueError` avec un message clair) puis ajoute l'identifiant du participant à l'ensemble de l'atelier. Un ensemble ignore le doublon si on inscrit deux fois la même personne.
  - `add_dependency(base, dependant)` ajoute `dependant` à l'ensemble `dependent_workshop_ids` de `base` . C'est une arête orientée : elle se lit « si `base` change, `dependant` est impacté ».

- `get_direct_participants` et `get_dependent_workshops` retournent une **copie** (`set(...)`), pour qu'un code extérieur ne puisse pas modifier le graphe par accident.

6.2 `src/data_loader.py` : du JSON au graphe

`load_event_graph(json_path)` ouvre le fichier, le parse avec `json.load`, crée un `EventGraph` vide puis le remplit dans cet ordre : participants, ateliers, inscriptions, dépendances. **L'ordre compte** : les inscriptions et dépendances vérifient que les identifiants existent déjà. `data.get('participants', [])` tolère une section absente du JSON au lieu de planter.

Pourquoi une fonction séparée de `EventGraph` ? Séparation des responsabilités : le graphe ne sait pas d'où viennent les données. Si demain elles viennent d'un CSV ou d'une base, on ne modifie que ce fichier.

6.3 `src/impact.py` : le coeur algorithmique

`ImpactResult` est un conteneur : `source_workshop_id`, `impacted_workshop_ids`, `impacted_participant_ids`. On le préfère à un tuple pour que `result.impacted_participant_ids` soit lisible.

`compute_impacted_set(graph, change_workshop_id)`, ligne par ligne :

1. **Garde-fou** : si l'atelier n'existe pas, `ValueError`.
2. **Initialisation** : `visited = {source}` et `queue = deque([source])`. `visited` est la variable la plus importante : elle empêche la boucle infinie en cas de cycle et le double comptage en cas de chemins multiples.
3. **Boucle BFS** : tant que la file n'est pas vide, `popleft()` sort l'atelier le plus anciennement ajouté (FIFO, ce qui définit un BFS). Pour chaque atelier qui en dépend, s'il n'est pas dans `visited`, on l'y ajoute et on le met en file.
4. **Collecte** : pour chaque atelier visité, `impacted_participants |= graph.get_direct_participants(...)`. L'opérateur `|=` est l'union d'ensembles : un participant inscrit à deux ateliers impactés n'apparaît qu'une fois.

Complexité : O(W + E + P), où W est le nombre d'ateliers atteints, E le nombre de dépendances parcourues et P les participants collectés. Chaque atelier est visité au plus une fois. Ce n'est pas un parcours brute-force $O(n^2)$. BFS ou DFS donneraient exactement le même ensemble : seul l'ordre de découverte change.

6.4 `src/notifications.py` : urgence et file de priorité

- **`Urgency`** est un `IntEnum` : ANNULATION = 0, CHANGEMENT_HORAIRE = 1, INFO_MINEURE = 2. Plus la valeur est basse, plus c'est urgent. Comme `heapq` sort toujours le **plus petit** élément, le plus urgent sort naturellement en premier.
- **`Notification`** : `participant_id`, `workshop_id`, `message`, `urgency`.
- **`NotificationQueue`** :
  - `_heap` est une liste gérée par `heapq` (un tas binaire). Chaque élément est un tuple `(urgence, compteur, notification)`.
  - `push` ajoute le tuple avec `heapq.heappush` ; `pop_next` retire le plus petit avec `heapq.heappop` et lève `IndexError` si la file est vide. Les deux sont en O(log n).
  - Le **compteur** (`itertools.count()`) sert de départage : deux notifications de même urgence sortent dans l'ordre d'arrivée. Sans lui, Python comparerait les objets `Notification` entre eux, ce qu'ils ne savent pas faire, et planterait.
  - `is_empty` et `__len__` (ce qui permet d'écrire `len(queue)`).
- **`build_notifications_from_impact`** fait le pont avec l'algorithme BFS : une notification par participant impacté, avec l'atelier source.

6.5 `src/rate_limiter.py` : le limiteur de débit

Idée : on mémorise l'heure des derniers envois dans une `deque` et on n'autorise un nouvel envoi que s'il y en a moins de N sur la dernière période.

- Le constructeur refuse `max_per_period <= 0` et `period_seconds <= 0` (`ValueError`).
- `_forget_expired(now)` : tant que le plus ancien envoi satisfait `envoi + période <= now`, on le retire. **Un envoi libère donc sa place exactement une période après lui**, c'est la fenêtre glissante. La frontière est fermée à gauche et ouverte à droite, comme dans le vérificateur (voir 6.8).
- `try_acquire()` : nettoie, puis si moins de N envois restent, mémorise l'heure actuelle et retourne `True`, sinon `False`. Il est appelé **au moment d'envoyer**.
- `seconds_until_available()` : 0 si une place est libre, sinon `plus ancien + période − maintenant`.
- Complexité : chaque envoi entre une fois et sort une fois, donc O(1) amorti, et la mémoire est bornée par N.

6.6 `src/dispatcher.py` : relier la file et le limiteur

- `SentNotification` (dataclass figée) : la notification et l'heure d'envoi, pour pouvoir vérifier le débit après coup.

- Le constructeur reçoit la file, le limiteur, `send_fn` (par défaut une fonction qui ne fait rien : l'envoi est simulé), `clock` et `sleep`.
- `dispatch_available()` : `while not queue.is_empty() and limiter.try_acquire()`. **L'ordre des deux conditions est volontaire** : on ne consomme une place du limiteur que s'il reste vraiment une notification à envoyer (test `test_empty_queue_consumes_no_rate_limit_slot`). Chaque tour sort la plus urgente, l'envoie et l'enregistre.
- `drain()` : répète `dispatch_available()` ; si la file n'est pas vide, il dort `seconds_until_available()` puis recommence, jusqu'à ce que la file soit vide.
- `_MIN_WAIT_SECONDS = 1e-6` : une attente minimale, au cas où une erreur d'arrondi de nombres décimaux donnerait une attente de 0,0000000001 s. Sans elle, l'horloge pourrait ne jamais avancer et la boucle tournerait à l'infini.

6.7 `src/batching.py` : la fusion des changements

- Constantes `STATUS_FIELD = 'statut'` et `CANCELLED_VALUE = 'annulé'` : la convention qui désigne une annulation.
- `RegisteredChange` : une trace (heure, atelier, champ, valeur, urgence, nombre d'impactés), pour le journal et les affichages.
- `_PendingGroup` : les changements en attente pour **un** participant. Son attribut `changes` est un dictionnaire `(atelier, champ) → (valeur, urgence)`. Réécrire une clé existante écrase l'ancienne valeur : c'est ainsi que **la dernière valeur gagne**.
- `ChangeBatcher(graph, window_seconds=30, clock)` : refuse une fenêtre négative. Il garde `_pending` (participant → groupe), `changes_log` et deux compteurs : notifications **sans fusion** et notifications **émises**. `notifications_saved` est leur différence.
- `register_change(workshop_id, field_name, new_value, urgency)` : calcule l'ensemble impacté avec le BFS, puis pour chaque participant crée un groupe (s'il n'existe pas, avec `opened_at = maintenant`) et y inscrit le changement. Le paramètre s'appelle `field_name` parce que `field` est déjà importé de `dataclasses`. Retourne le nombre d'impactés.
- `flush_ready()` : émet les groupes dont `maintenant − opened_at >= window`. `flush_all()` force tout. `_flush` retire les groupes de `_pending` et incrémente le compteur d'émis.
- `_build_notification(participant, groupe)`, en trois temps :
  1. **Annulations** : l'ensemble des ateliers dont la clé `statut` vaut `annulé`. Les autres changements de ces ateliers sont retirés ; la clé `statut` est conservée.
  2. **Urgence** : `min(...)` des urgences restantes, donc la plus haute.

3. **Message** : regroupé par atelier, `Atelier : champ → valeur, champ → valeur`, ateliers séparés par une barre verticale. Un `statut` s'écrit juste avec sa valeur (`annulé`). L'atelier principal de la notification est le premier atelier du message.

6.8 `clock.py`, `report.py`, `batching_report.py`

- **`SimulatedClock`** : `now`, `__call__` (permet d'écrire `clock()`), `advance(secondes)` (refuse de reculer) et `sleep` (même signature que `time.sleep`, mais fait seulement avancer l'horloge).
- **`max_in_any_window(timestamps, period)`** : le **vérificateur indépendant**. Il trie les heures puis, pour chaque envoi pris comme début de fenêtre, compte combien d'envois tombent dans `[début, début + période[` avec deux pointeurs (O(n) après le tri). Il retourne le maximum. Il ne connaît pas le limiteur : s'il y avait un bug, il le verrait.
- **`render_send_histogram`** : regroupe les envois par tranche de `période` secondes (un `Counter`), dessine une barre par tranche, puis affiche le maximum observé et le verdict OK ou LIMITE DÉPASSÉE. **`render_send_order`** liste les premiers envois avec leur heure et leur urgence.
- **`render_batching_report`** (dans `batching_report.py`) : affiche les changements reçus, les notifications émises et le bilan `N sans fusion → M émises (K évitées)`.

6.9 Configuration

- **`tests/conftest.py`** : deux *fixtures*, `clock` (une horloge simulée neuve) et `sample_graph` (l'événement de test chargé). Un test les reçoit simplement en les citant en paramètre.
- **`pytest.ini`** : `testpaths = tests` et `addopts = -v -rP`.
- **`requirements.txt`** : `pytest` et `ruff`. **`.gitignore`** : `venv/`, `__pycache__/`, `.pytest_cache/`.

7. Le détail de chaque test

7.0 Les principes communs

- **Arrange, Act, Assert.** Chaque test construit une petite situation (Arrange), appelle la fonction testée (Act), puis vérifie le résultat avec `assert` (Assert). Si on te demande d'expliquer un test au hasard : dis **quel cas précis** il vérifie et **ce qui casserait** s'il échouait.
- **Fixtures** (`clock`, `sample_graph`) : fournies automatiquement par `conftest.py` quand le test les cite en paramètre.

- **`parametrize`** : un même test lancé avec plusieurs jeux de valeurs, compté comme autant de tests.
- **`pytest.raises(ValueError)`** : le test réussit si l'erreur attendue est bien levée. **`pytest.approx`** : comparaison de nombres décimaux avec une petite tolérance.

Répartition : graph 3, data_loader 2, impact 7, notifications 5, rate_limiter 11, dispatcher 11, batching 18, pipeline 1, clock 5, simulation 14, cli 15. **Total : 92.**

7.1 `test_graph.py` (Tâche 1, 3 tests)

Test	Ce qu'il fait
`test_add_participant_and_workshop`	Ajoute Alice (p1) et un atelier (w1), vérifie qu'ils sont dans les deux dictionnaires
`test_enroll_participant`	Inscrit p1 à w1 et vérifie que `get_direct_participants('w1')` vaut {p1}
`test_dependency_between_workshops`	Déclare que w2 dépend de w1 et vérifie que `get_dependent_workshops('w1')` vaut {w2}

7.2 `test_data_loader.py` (Tâche 2, 2 tests)

Test	Ce qu'il fait
`test_load_sample_event`	Charge le JSON : 5 participants, 4 ateliers, w2 dépend de w1, les inscrits de w1 sont {p1, p2}
`test_load_sample_event_dependencies_are_preserved`	Vérifie la seconde chaîne : w4 dépend de w3 (le chargeur ne perd pas de dépendance)

7.3 `test_impact.py` (Tâches 3 et 4, 7 tests)

Test	Situation construite	Ce qu'il vérifie
`test_no_impact_when_no_dependency`	Un atelier seul avec un inscrit	Seul cet atelier et cette personne sont impactés : **absence de cascade**

Test	Situation construite	Ce qu'il vérifie
`test_transitive_cascade_a_depends_b_depends_c`	Chaîne w1 → w2 → w3, un inscrit par atelier	Partir de w1 impacte les 3 ateliers et les 3 personnes : **cascade profonde**
`test_no_duplicates_on_diamond_shaped_dependencies`	Losange : w1 → w2, w1 → w3, w2 → w4, w3 → w4, un inscrit en w4	w4 est atteint par deux chemins mais compté une seule fois : **pas de doublons**
`test_cycle_does_not_cause_infinite_loop`	w1 dépend de w2 qui dépend de w1	Le calcul se termine et donne {w1, w2} : **cycle géré par `visited`**
`test_unknown_workshop_raises`	Un atelier inexistant	`ValueError` : **garde-fou**
`test_large_graph_performance_and_correctness`	Chaîne de 300 ateliers, un inscrit chacun	Les 300 ateliers et 300 personnes sont impactés, et le calcul dure moins d'une seconde (mesuré : environ 0,14 ms). Le seuil est volontairement large : c'est un test de non-régression, pas un benchmark
`test_on_sample_event_dataset`	Le vrai jeu de données	Cuisine impacte {w1, w2} et les personnes {p1, p2, p3} : **cas concret de bout en bout**

7.4 `test_notifications.py` (Tâche 5, 5 tests)

Test	Ce qu'il fait
`test_single_push_pop`	Une notification entre, la file en contient une, la même (`is`) ressort, la file est vide
`test_most_urgent_notification_comes_out_first`	On pousse volontairement dans le désordre : info mineure, annulation, changement d'horaire. Ils ressortent dans l'ordre annulation, changement, info. **C'est le test de la DoD de la Tâche 5**
`test_same_urgency_keeps_insertion_order`	Deux annulations : la première entrée sort la première. Prouve le rôle du compteur de départage
`test_pop_from_empty_queue_raises`	Retirer dans une file vide lève `IndexError`
`test_build_notifications_from_impact`	À partir d'un résultat d'impact de 3 personnes : 3 notifications, bons identifiants, bonne urgence, atelier source

7.5 `test_rate_limiter.py` (Tâche 6, 9 tests, puis 2 de plus en Tâche 8 : voir la section 11)

Test	Ce qu'il fait
`test_allows_up_to_the_limit_then_blocks`	Limite 3 par seconde, 5 demandes au même instant : résultat `True, True, True, False, False`
`test_slots_are_freed_once_the_period_has_elapsed`	Limite 2. Après 2 envois, un troisième est refusé ; à 0,99 s il l'est encore ; à 1,0 s il passe. **Teste la frontière exacte**
`test_seconds_until_available`	Limite 1 par 2 s : 0 au départ ; après un envoi et 0,5 s d'attente, il reste 1,5 s

Test	Ce qu'il fait
`test_window_slides_instead_of_resetting`	Limite 2. Envois à t = 0 et t = 0,6. À t = 1,0 une seule place s'est libérée (celle de t = 0), pas deux. **Prouve que la fenêtre glisse** au lieu de se réinitialiser à intervalle fixe
`test_invalid_configuration_is_rejected` (4 cas)	Limite 0, limite négative, période 0, période négative : `ValueError` dans chaque cas
`test_independent_checker_detects_violations`	On donne au vérificateur un comportement fautif connu (4 envois en 0,3 s) : il doit compter 4. Et 3 envois bien espacés : il doit compter 1. **Prouve que le vérificateur lui-même est fiable**

7.6 `test_dispatcher.py` (Tâche 6, 11 tests)

Un utilitaire `fill_queue(n)` remplit la file avec des urgences mélangées : une sur dix est une annulation, trois sur dix un changement d'horaire, le reste des infos mineures. Pour 120 notifications : 12 annulations, 36 changements d'horaire, 72 infos.

Test	Ce qu'il fait
`test_limit_is_never_exceeded_with_120_notifications`	**Test principal de la DoD.** 120 notifications, limite 5 par seconde. Vérifie que les 120 partent et que le maximum observé sur n'importe quelle fenêtre d'une seconde est au plus 5. Affiche l'histogramme
`test_urgent_notifications_leave_first_even_under_rate_limit`	Même situation : la liste des urgences dans l'ordre d'envoi est triée, et les 12 annulations partent en premier. Affiche les 14 premiers envois
`test_total_duration_matches_the_configured_rate`	120 envois à 5 par seconde = 24 paquets de 5 : le premier part à t = 0, le dernier à t = 23

Test	Ce qu'il fait
`test_limit_respected_for_many_configurations` (5 cas)	Même vérification pour (5 par 1 s, 120), (10 par 1 s, 150), (3 par 0,5 s, 101), (50 par 60 s, 400), (1 par 2 s, 30). Tout part et la limite tient dans chaque configuration
`test_dispatch_available_only_sends_what_the_limiter_allows`	20 en file, limite 5 : 5 partent, 15 restent ; un deuxième appel n'envoie rien ; après 1 s d'horloge, 5 de plus
`test_empty_queue_consumes_no_rate_limit_slot`	Avec une file vide, `dispatch_available` n'envoie rien **et** ne consomme pas la seule place du limiteur. Prouve l'ordre des deux conditions
`test_send_function_is_called_for_each_notification`	Une fonction d'envoi personnalisée est appelée 12 fois pour 12 notifications : l'envoi est bien remplaçable

7.7 `test_batching.py` (Tâche 7, 15 tests, puis 3 de plus en Tâche 8 : voir la section 11)

Rappel : un changement sur w1 impacte Alice, Bruno et Chloe ; un changement sur w3 impacte David et Emma.

Test	Ce qu'il fait
`test_three_close_changes_give_one_final_coherent_notification`	**Test principal de la DoD.** Sur w1 : horaire 15h à t = 0, horaire 16h à t = 5, salle B à t = 12. À t = 12 la fenêtre n'est pas écoulée : rien n'est émis. À t = 30 : exactement 3 notifications (une par personne), chacune contient 16h et `salle → B`, aucune ne contient 15h, urgence CHANGEMENT_HORAIRE
`test_without_batching_there_would_be_nine_notifications`	3 changements × 3 personnes = 9 sans fusion ; 3 émises ; 6 évitées

Test	Ce qu'il fait
`test_changes_far_apart_are_not_merged`	Un changement, attente de 31 s, émission (3 notifications) ; un second changement, nouvelle attente, nouvelle émission : 2 lots distincts, le premier contient 15h, le second 16h
`test_window_is_configurable` (5 cas)	Fenêtre 10 s : pas prêt à 9,9 s, prêt à 10 s. Fenêtre 60 s : pas prêt à 30 s, prêt à 60 s. Fenêtre 0 : prêt tout de suite
`test_cancellation_supersedes_other_changes_of_the_same_workshop`	Poterie : horaire 15h puis annulation. David et Emma reçoivent un message qui contient `annulé`, ne contient pas `horaire`, en urgence ANNULATION. **Aucune information contradictoire**
`test_cancellation_then_reinstatement_keeps_only_the_final_state`	Annulation puis, 5 s plus tard, `maintenu` : le message final dit `maintenu`, pas `annulé`, et l'urgence redescend à INFO_MINEURE
`test_participant_in_two_changed_workshops_gets_a_single_notification`	Bruno est inscrit à w1 et w2. Changement d'horaire sur w1 et de salle sur w2 : 3 notifications. Alice (seulement w1) n'entend pas parler de la salle ; Bruno reçoit **les deux infos dans un seul message**
`test_independent_changes_do_not_interfere`	Un changement sur w1 et un sur w3 : 5 notifications. Alice voit 15h et pas 10h ; David voit 10h et pas 15h
`test_flush_all_emits_everything_immediately`	Fenêtre de 300 s : `flush_ready` ne donne rien, `flush_all` donne les 3 notifications, plus rien en attente
`test_unknown_workshop_is_rejected`	Un atelier inconnu lève `ValueError`
`test_negative_window_is_rejected`	Une fenêtre négative lève `ValueError`

7.8 `test_pipeline.py` (intégration, 1 test)

`test_full_pipeline_from_changes_to_rate_limited_sending` fait tourner **toute la chaîne**. Fenêtre 30 s, limite 2 par seconde. Sur w1 : horaire 15h à t = 0 puis 16h à t = 5 ; annulation de w3 à t = 10 ; attente jusqu'à t = 40 ; émission, mise en file, envoi. Il vérifie :

- 5 notifications envoyées, pour 5 personnes **distinctes** (une seule chacune) ;
- les urgences sont triées, et les deux premières envoyées sont David et Emma (annulation) ;
- le débit maximal observé est au plus 2.

Il affiche le rapport de fusion, l'ordre d'envoi et l'histogramme : c'est le test le plus parlant à montrer en direct.

8. Les questions pièges et les réponses

Et si le graphe contient un cycle ? L'ensemble `visited` mémorise chaque atelier dès qu'il est vu. Un atelier déjà vu n'est jamais remis en file, donc le calcul se termine toujours. Test : `test_cycle_does_not_cause_infinite_loop`.

Pourquoi BFS et pas DFS ? Le résultat est identique : le même ensemble d'ateliers et de personnes, seul l'ordre de découverte change. BFS utilise une file (premier arrivé, premier sorti) et explore niveau par niveau ; DFS utiliserait une pile (dernier arrivé, premier sorti). Dans le code, remplacer `popleft()` par `pop()` transformerait le BFS en DFS. Le sujet acceptait les deux.

Pourquoi une `deque` et pas une liste ? `list.pop(0)` doit décaler tous les éléments : O(n). `deque.popleft()` est en O(1).

Pourquoi des ensembles (`set`) partout ? Le test d'appartenance (`x in s`) est en O(1) et les doublons sont impossibles. C'est ce qui rend le BFS linéaire et ce qui garantit qu'une personne n'est comptée qu'une fois.

Pourquoi `heapq` plutôt qu'une liste triée ? Un tas binaire ajoute et retire en O(log n). Retrier une liste à chaque ajout coûterait O(n log n) à chaque fois. Et `heapq` est dans la bibliothèque standard : rien à maintenir.

Pourquoi un compteur dans le tuple du tas ? Deux notifications de même urgence se retrouveraient à égalité sur le premier élément du tuple, et Python tenterait de comparer les objets `Notification`, ce qui lève une erreur. Le compteur départage et garantit l'ordre d'arrivée.

Fenêtre glissante ou token bucket ? Le token bucket autorise des rafales : un seau plein de N jetons laisse partir N envois d'un coup, puis se recharge en continu, donc sur une même seconde on peut dépasser N. La fenêtre glissante garantit strictement qu'il n'y a

jamais plus de N envois sur n'importe quelle période, ce qui est exactement ce que la DoD demande de tester. Le sujet autorisait « token bucket ou équivalent ».

Comment prouves-tu que la limite n'est jamais dépassée ? Trois niveaux : un vérificateur indépendant (`max_in_any_window`) qui recompte les envois sans connaître le limiteur ; ce vérificateur est lui-même testé sur un comportement fautif connu ; et la vérification est rejouée sur 5 configurations différentes (jusqu'à 400 notifications). Je peux aussi casser le limiteur devant vous pour montrer que les tests deviennent rouges (section 2.5).

Pourquoi une horloge simulée ? Avec de vraies attentes, tester une fenêtre de 30 secondes prendrait 30 secondes et donnerait des résultats instables. Avec l'horloge simulée, les 92 tests s'exécutent en une fraction de seconde et donnent toujours le même résultat.

Les notifications sont-elles vraiment envoyées ? Non, l'envoi est simulé. `send_fn` est un point d'extension : on y brancherait un vrai appel SMS, email ou push sans toucher au reste.

Quelle est la complexité ? BFS : O(W + E + P). Ajout et retrait dans la file : O(log n). Limiteur : O(1) amorti par envoi, mémoire bornée par N. Enregistrer un changement : un BFS plus une opération par personne impactée.

Et si un changement arrive juste après l'émission d'un groupe ? Au moment de l'émission, le groupe est retiré de `_pending`. Le nouveau changement ouvre un nouveau groupe, donc la personne recevra une seconde notification. C'est voulu : l'information de la première est déjà partie (test `test_changes_far_apart_are_not_merged`).

Qui déclenche l'émission quand la fenêtre est écoulée ? Personne automatiquement : `flush_ready()` doit être appelé par le code qui utilise le batcher (une boucle, un minuteur). C'est une limite assumée (section 9).

Dans quel ordre partent deux personnes de même urgence ? Dans l'ordre où elles ont été mises en file. Mais cet ordre vient du parcours d'un ensemble, que Python ne garantit pas identique d'une exécution à l'autre. Aucun test ne dépend de cet ordre.

La cascade marche-t-elle dans les deux sens ? Non. La dépendance est orientée : un changement sur Dégustation n'impacte pas Cuisine.

Pourquoi le seuil du test de performance est-il large (1 seconde) ? Pour ne pas casser sur une machine lente ou sur la CI. Mesuré : environ 0,14 ms. C'est un test de non-régression qui détecterait un comportement quadratique, pas un benchmark.

9. Limites assumées et pistes d'amélioration

Mieux vaut les dire toi-même que de te les faire découvrir. Une phrase pour les introduire : « Voici ce que je n'ai volontairement pas fait, et comment je le ferais. »

- **Pas d'ordonnanceur.** Le regroupement fonctionne par appel : il faut appeler `flush_ready()` régulièrement. Dans la simulation (src/simulation.py, Tâche 8), c'est la boucle d'événements qui le fait ; en production, un minuteur s'en chargerait.
- **L'urgence n'accélère pas l'émission.** Une annulation attend la fin de la fenêtre de 30 s comme une info mineure. Piste : émettre immédiatement les ANNULATION, ou raccourcir la fenêtre selon l'urgence.
- **Pas de concurrence.** Le batcher, la file et le limiteur ne sont pas sûrs en cas d'utilisation par plusieurs threads.
- **Envoi simulé, sans réessai** si un envoi échouait.
- **Tout en mémoire** : pas de sauvegarde ; le journal `changes_log` grandit sans limite.
- **Fusion simple** : la dernière valeur gagne. Pas de gestion de plusieurs organisateurs qui modifient en même temps. La convention `statut` / `annulé` est en dur.
- **Une seule sorte de dépendance**, sans poids ni limite de profondeur de cascade.

10. Check-list avant la soutenance

1. `git push` fait, onglet Actions de GitHub **vert**, cartes des tâches 1 à 8 en **Done** sur le Project.
2. Lancer une fois `python3 -m pytest` : `92 passed`. Lancer `ruff check .` : `All checks passed!`.
3. Un terminal déjà ouvert dans le dossier du projet, prêt à lancer `python3 demo_pipeline.py` et `python3 demo.py w1`.
4. Pouvoir expliquer sans notes : le BFS et `visited`, `heapq` et le compteur, la fenêtre glissante, les règles de fusion, et pourquoi l'horloge est simulée.
5. Pouvoir dire les limites de la section 9 avant qu'on te les demande.

11. Mise à jour : ce que la Tâche 8 a ajouté

Le projet compte maintenant **92 tests** (53 + 39 nouveaux). Cette section décrit les ajouts ; le reste du guide reste valable.

11.1 Nouveaux fichiers

Fichier	Rôle
`cli.py`	CLI de démonstration : prend des changements en entrée, affiche le résultat de bout en bout

Fichier	Rôle
`src/simulation.py`	Moteur de simulation : tout se déroule sur une seule ligne de temps simulée
`src/impact_report.py`	Affichage de l'étape 1 : qui est impacté par chaque changement
`README.md`	Installation, usage, architecture, tests, limites
`docs/SOUTENANCE.md`	Démo live scriptée : commandes exactes et phrases à dire
`tests/test_clock.py`, `test_simulation.py`, `test_cli.py`	5, 14 et 15 tests

11.2 Ce qui a changé dans les modules existants (ajouts seulement)

- **`clock.py`** : `advance_to(moment)` place l'horloge exactement à un instant, sans l'erreur d'arrondi d'une addition de nombres décimaux.
- **`rate_limiter.py`** : `available_at()` donne l'heure absolue à partir de laquelle un envoi sera possible.
- **`batching.py`** : `next_flush_time()` donne l'heure du prochain groupe prêt. Et `flush_ready` compare maintenant `now >= opened_at + window`, le même calcul que `next_flush_time`, ce qui évite qu'un arrondi décimal rende un groupe « presque prêt ».
- **`report.py`** : l'histogramme résume une longue série de tranches vides en une seule ligne (au lieu de 13 lignes vides).
- **`batching_report.py`** : `render_emissions` affiche l'heure de fin de chaque fenêtre.

11.3 La simulation événementielle, pas à pas

Dans un vrai système, quelque chose doit émettre un groupe quand sa fenêtre se termine et envoyer dès que le limiteur le permet. `run_simulation` joue ce rôle : elle saute d'événement en événement dans l'ordre du temps. Il y a trois sortes d'événements : un changement arrive, une fenêtre de regroupement se termine, le limiteur libère une place.

1. Les changements sont triés par heure (tri stable) et l'urgence est déduite par `infer_urgency` si elle n'est pas donnée : champ `statut` avec la valeur `annulé` = ANNULATION, champ `horaire` = CHANGEMENT_HORAIRE, sinon INFO_MINEURE.
2. L'impact de chaque changement est calculé d'abord : un atelier inconnu lève `ValueError` dès le départ.

3. Boucle : on calcule les prochaines heures où quelque chose se passe (prochain changement, prochaine fin de fenêtre, prochaine place libre du limiteur), on avance l'horloge à la plus proche, puis on fait **dans cet ordre** : émettre les groupes dont la fenêtre est finie, enregistrer les changements qui arrivent, envoyer ce que le limiteur autorise.
4. Règle à égalité de temps : un changement qui arrive **pile** à la fin d'une fenêtre appartient au groupe suivant (test `test_change_exactly_at_the_deadline_goes_to_the_next_group`).
5. Garde-fou : si quatre tours de suite ne font avancer rien, la simulation lève `RuntimeError` au lieu de tourner sans fin.

Pourquoi c'est important : sans cet ordonnanceur, un changement à t = 100 s aurait été fusionné à tort avec un changement à t = 0 dont la fenêtre de 30 s était finie depuis longtemps.

11.4 La CLI

`python3 cli.py --change HEURE ATELIER CHAMP VALEUR` (répétable), plus `--window`, `--limit`, `--period`, `--data`, `--rows`, `--list`. Sans `--change`, un scénario de démonstration est joué. Elle affiche quatre étapes : qui est impacté (BFS), la fusion, l'ordre d'envoi, le respect du débit.

- `main(argv)` retourne un code de sortie au lieu de quitter : c'est ce qui la rend testable directement.
- Toute entrée invalide (atelier inconnu, heure non numérique ou négative, fenêtre négative, limite nulle, fichier absent) donne un message clair sur la sortie d'erreur, le code 2, et **rien** sur la sortie normale.
- Le scénario par défaut place l'annulation de Poterie à t = 0, dans la même fenêtre que le premier changement de Cuisine : les deux groupes sont émis ensemble, donc on voit la fusion (11 vers 5), la priorité (annulations d'abord) et le débit (2 par seconde).

11.5 Le détail des 39 nouveaux tests

`test_clock.py` (5 tests)

Test	Ce qu'il fait
`test_clock_starts_at_zero_and_is_callable`	Une horloge neuve vaut 0 et s'appelle comme une fonction
`test_advance_and_sleep_move_time_forward`	2,5 s d'`advance` plus 1,5 s de `sleep` donnent 4,0

Test	Ce qu'il fait
`test_advance_to_sets_the_exact_moment`	`advance_to(0.1 + 0.3)` donne exactement cette valeur
`test_time_cannot_go_backwards` (2 cas)	`advance(-1)` et `advance_to` vers le passé lèvent `ValueError`

Ajouts à `test_rate_limiter.py` (2 tests) et `test_batching.py` (3 tests)

Test	Ce qu'il fait
`test_available_at_is_now_when_a_slot_is_free`	Avec une place libre, l'heure disponible est l'heure actuelle
`test_available_at_gives_the_absolute_time_when_blocked`	Limite 1 par 2 s, envoi à t = 1 : disponible à t = 3 ; sauter exactement à 3 suffit pour envoyer
`test_next_flush_time_is_none_when_nothing_is_pending`	Rien en attente : pas de prochaine émission
`test_next_flush_time_is_the_earliest_group_plus_the_window`	Groupes ouverts à t = 3 et t = 10, fenêtre 30 : prochaine émission à 33, puis 40
`test_flush_is_exact_at_the_deadline_even_with_decimal_times`	Fenêtre 0,3 et changement à 0,1 : le groupe est prêt à l'instant exact calculé, malgré les arrondis

`test_simulation.py` (14 tests)

Test	Ce qu'il fait
`test_urgency_is_inferred_from_the_change` (4 cas)	statut + annulé = ANNULATION ; statut + maintenu = INFO_MINEURE ; horaire = CHANGEMENT_HORAIRE ; salle = INFO_MINEURE
`test_explicit_urgency_is_respected`	Une urgence ANNULATION donnée sur un champ `horaire` est conservée. **Ce test a trouvé un vrai bug** (voir 11.6)
`test_single_change_is_emitted_when_its_window_ends`	Un changement à t = 0 : une émission à t = 30, 3 envois, tous à t = 30 (limite 5)

Test	Ce qu'il fait
`test_impact_is_computed_for_each_change`	Cuisine impacte {w1, w2} et {p1, p2, p3}
`test_close_changes_are_merged_into_one_notification_each`	3 changements : 9 sans fusion, 3 envoyées, 6 évitées, toutes avec 16h
`test_late_change_is_not_merged_with_an_expired_window`	Changements à t = 0 et t = 100 : deux émissions, à 30 et 130 ; Alice est prévenue deux fois, 15h puis 16h. **Teste l'ordonnanceur**
`test_change_exactly_at_the_deadline_goes_to_the_next_group`	Changements à t = 0 et t = 30 : émissions à 30 et 60
`test_rate_limit_spreads_the_sending_over_time`	5 personnes, fenêtre 10, limite 1 par seconde : envois à 10, 11, 12, 13, 14
`test_urgent_notifications_leave_first_when_emitted_together`	Changement d'horaire et annulation émis ensemble, limite 1 : les deux premiers envois sont les annulations (David et Emma)
`test_changes_are_processed_in_time_order_whatever_the_input_order`	Changements donnés dans le désordre : triés par heure, et 16h gagne
`test_unknown_workshop_is_rejected`	Atelier inconnu : `ValueError`

`test_cli.py` (15 tests)

Test	Ce qu'il fait
`test_default_scenario_shows_the_four_steps`	Code 0, les 4 étapes, « 11 notifications sans fusion -> 5 émises (6 évitées) », verdict OK
`test_single_change_shows_who_is_impacted_and_who_is_spared`	Un changement sur Cuisine : « à notifier (3/5) : Alice, Bruno, Chloe », « épargnés (2) : David, Emma », cascade visible
`test_merging_is_visible_in_the_output`	3 changements : « 9 sans fusion -> 3 émises (6 évitées) » et le message `horaire → 16h, salle → B`

Test	Ce qu'il fait
`test_priority_is_visible_in_the_send_order`	Dans l'étape 3, ANNULATION apparaît avant CHANGEMENT_HORAIRE
`test_rate_limit_options_are_applied`	`--window 0 --limit 1` : l'affichage dit « limite 1 par 1s » et le maximum observé est 1
`test_list_shows_workshops_and_participants`	`--list` montre w1, Atelier Cuisine, Alice
`test_help_describes_usage_and_exits_cleanly`	`--help` quitte avec le code 0 et affiche des exemples
`test_invalid_input_gives_a_clear_error_and_exit_code_2` (6 cas)	Atelier inconnu, heure `abc`, heure négative, fenêtre négative, limite 0, fichier absent : code 2, message sur la sortie d'erreur, rien sur la sortie normale
`test_unknown_workshop_error_lists_the_valid_ones`	Le message d'erreur cite les ateliers valides (w1 à w4)
`test_script_is_really_executable_from_the_command_line`	Lance `cli.py` dans un vrai processus Python séparé : code 0, étape 4 affichée, aucune erreur

11.6 Trois trouvailles à raconter en soutenance

1. **Un bug attrapé par un test.** J'écrivais `change.urgency or infer_urgency(...)`. Or ANNULATION vaut 0, et 0 est « faux » en Python : une annulation explicite était écrasée. Correctif : `is not None`. C'est la raison pour laquelle `test_explicit_urgency_is_respected` existe.
2. **La simulation a révélé la limite de la section 9.** Avec `python3 cli.py --change 0 w1 horaire 15h --change 15 w3 statut annulé`, l'annulation arrive plus tard : elle attend sa propre fenêtre et part à t = 45, **après** le changement d'horaire parti à t = 30. La priorité ne joue que sur ce qui est dans la file en même temps. Piste d'amélioration : émettre immédiatement les annulations.
3. **Le code a été cassé volontairement pour valider les tests.** Ordonnanceur qui ignore les fins de fenêtre : 10 tests échouent. Changements enregistrés avant l'émission à égalité de temps : 1 test échoue. Limiteur ignoré : la simulation tournait sans fin, d'où le garde-fou qui la transforme en erreur claire (2 tests échouent).

11.7 Pour la soutenance

Le script exact (commandes, ce qu'on voit, phrases à dire, plan B) est dans `docs/SOUTENANCE.md`. Le README (`README.md`) à la racine du dépôt donne l'installation, l'usage et l'architecture.